

Class 12: Transcriptomics and RNAseq Analysis

Seona Patel (PID: A69035519)

Table of contents

1. Background	1
Data Import	2
Toy analysis	4
Filter out zero count genes	9
DESeq Analysis	10
Save our results	12
Volcano Plot	12
Adding annotations	13
Pathway Analysis	17

1. Background

Today we will analyze soem RNAseq data from Himes et al. on the effects of a common steroid dexamethasone (dex) on airway smooth muscle cells (ASMs).

The data for this hands-on session comes from a published RNA-seq experiment where airway smooth muscle cells were treated with dexamethasone, a synthetic glucocorticoid steroid with anti-inflammatory effects (Himes et al. 2014).

Glucocorticoids are used, for example, by people with asthma to reduce inflammation of the airways (Figure 1). The anti-inflammatory effects on airway smooth muscle (ASM) cells has been known for some time but the underlying molecular mechanisms are unclear.

Himes et al. used RNA-seq to profile gene expression changes in four different ASM cell lines treated with dexamethasone glucocorticoid.

They found a number of differentially expressed genes but focus much of the discussion on a gene called CRISPLD2.

This gene encodes a secreted protein known to be involved in lung development, and SNPs in this gene in previous GWAS studies are associated with inhaled corticosteroid resistance and bronchodilator response in asthma patients.

For this analysis, we need 2 main inputs:

- **countData**: a table of **counts** per gene (in rows) across experiments (in columns)
- **colData**: **metadata** about the design of the experiments. The rows here must match the columns in **countData**

Data Import

```
counts <- read.csv("airway_scaledcounts.csv", row.names=1)
metadata <- read.csv("airway_metadata.csv")
```

Let's have a wee peak at our **counts** data

```
head(counts)
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG000000000003	723	486	904	445	1170
ENSG000000000005	0	0	0	0	0
ENSG000000000419	467	523	616	371	582
ENSG000000000457	347	258	364	237	318
ENSG000000000460	96	81	73	66	118
ENSG000000000938	0	0	1	0	2
	SRR1039517	SRR1039520	SRR1039521		
ENSG000000000003	1097	806	604		
ENSG000000000005	0	0	0		
ENSG000000000419	781	417	509		
ENSG000000000457	447	330	324		
ENSG000000000460	94	102	74		
ENSG000000000938	0	0	0		

The IDs are for each gene in the ensemble database.

```
metadata
```

	id	dex	celltype	geo_id
1	SRR1039508	control	N61311	GSM1275862
2	SRR1039509	treated	N61311	GSM1275863
3	SRR1039512	control	N052611	GSM1275866
4	SRR1039513	treated	N052611	GSM1275867
5	SRR1039516	control	N080611	GSM1275870
6	SRR1039517	treated	N080611	GSM1275871
7	SRR1039520	control	N061011	GSM1275874
8	SRR1039521	treated	N061011	GSM1275875

Note that the rows match the columns in counts.

Q1. How many “genes” are in this dataset?

```
nrow(counts)
```

```
[1] 38694
```

There are 38694 genes in the dataset.

Q2. How many experiments (i.e. columns in `counts` or rows in `metadata`) are there?

```
ncol(counts)
```

```
[1] 8
```

There are 8 experiments.

Q3. How many control experiments?

```
sum(metadata$dex == "control")
```

```
[1] 4
```

There are 4 control experiments.

Toy analysis

1. Extract the “control” columns from counts
2. Calculate the mean value for each gene in “control” columns 3-4. Do the same for the “treated” columns
3. Compare these mean values for each gene

Step 1:

```
control.inds <- metadata$dex == "control"
control.counts <- counts[,control.inds]
```

Step 2:

```
control.mean <- rowMeans(control.counts)
```

Step 3:

```
treated.inds <- metadata$dex == "treated"
treated.counts <- counts[,treated.inds]
```

Step 4:

```
treated.mean <- rowMeans(treated.counts)
```

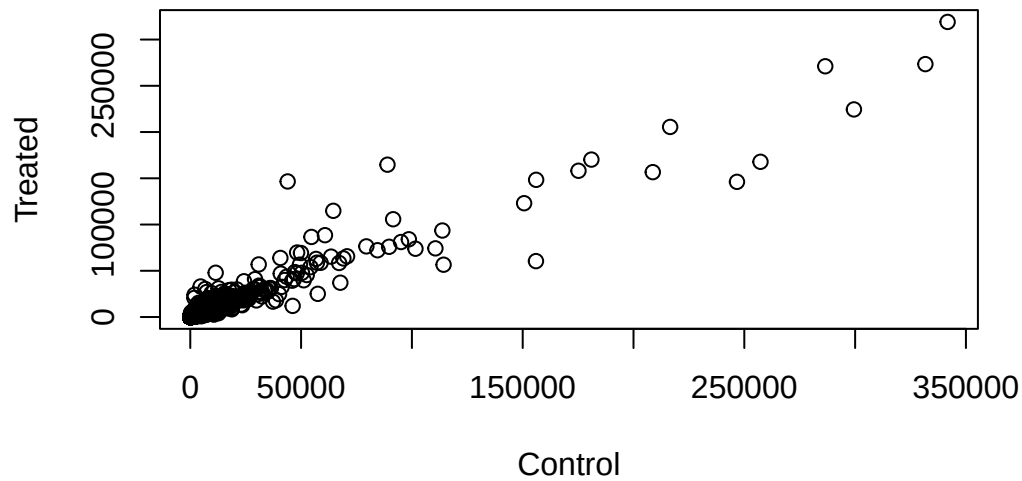
Step 5:

```
meancounts <- data.frame(control.mean, treated.mean)
head(meancounts)
```

	control.mean	treated.mean
ENSG000000000003	900.75	658.00
ENSG000000000005	0.00	0.00
ENSG000000000419	520.50	546.00
ENSG000000000457	339.75	316.50
ENSG000000000460	97.25	78.75
ENSG000000000938	0.75	0.00

Q5 Create a scatter plot showing the mean of the treated samples against the mean of the control samples. You could also use

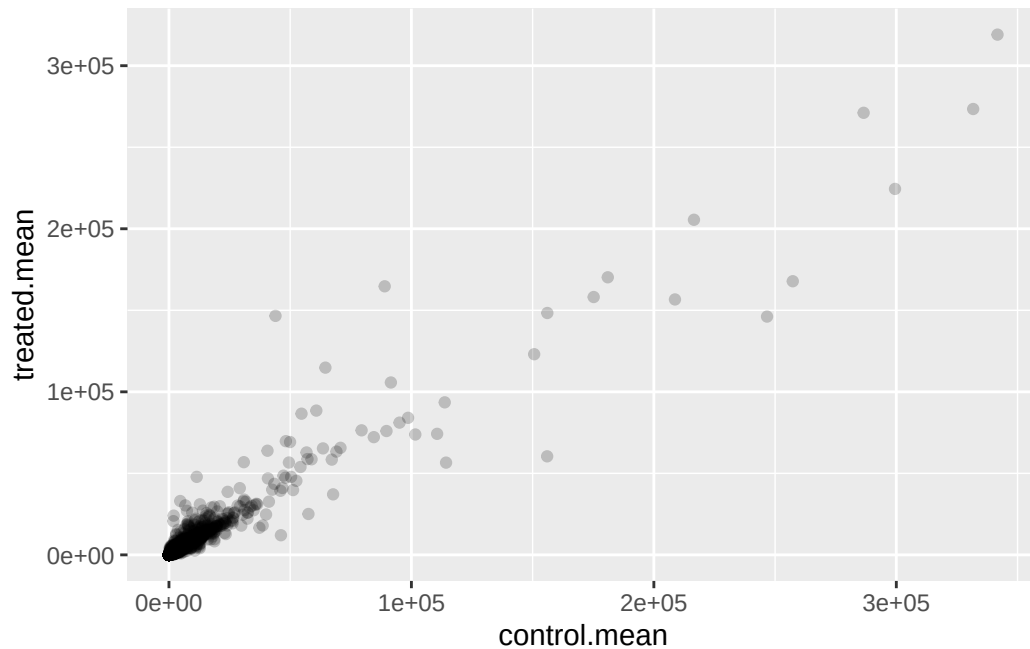
```
plot(meancounts[,1], meancounts[,2], xlab="Control", ylab="Treated")
```



Q5b. You could also use ggplot2 package to make this figure producing the plot below. What `geom_?()` function would you use for this plot?

I would use `geom_point()`

```
# in base R: plot(meancounts)
library(ggplot2)
ggplot(meancounts) + aes(control.mean, treated.mean) +
  geom_point(alpha=0.2)
```



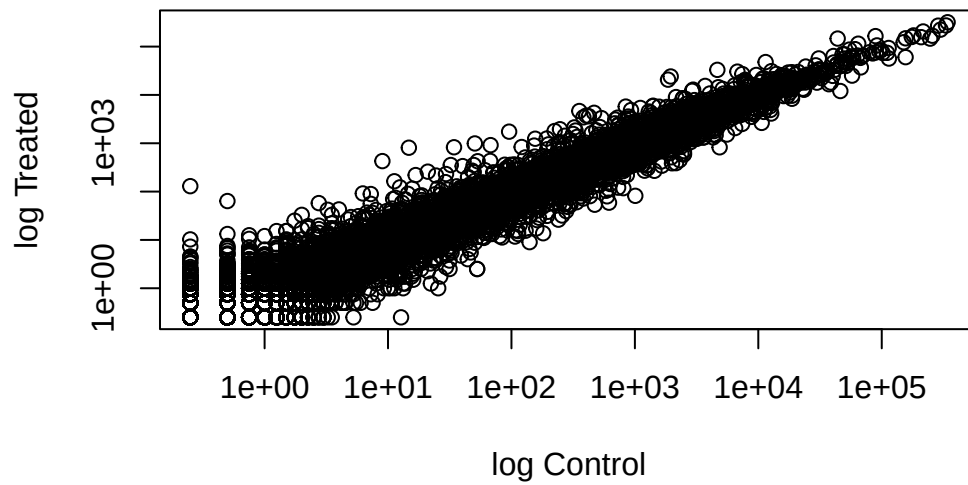
This graph is super skewed, need to log it.

Q6. Try plotting both axes on a log scale. What is the argument to `plot()` that allows you to do this?

```
plot(meancounts[,1], meancounts[,2], log="xy", xlab="log Control", ylab="log Treated")
```

Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15032 x values ≤ 0 omitted from logarithmic plot

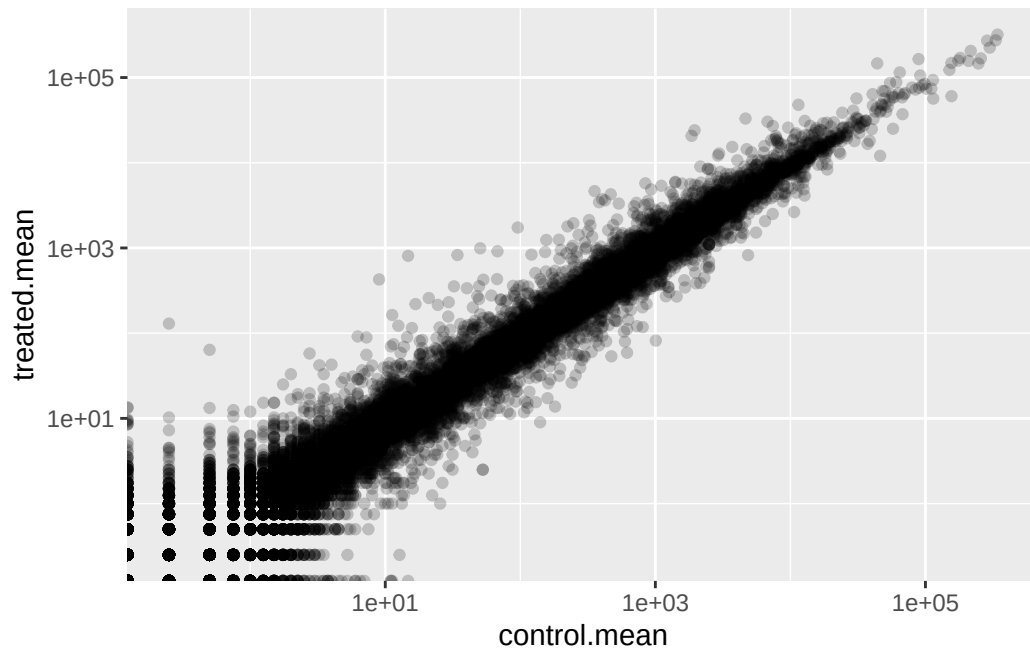
Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15281 y values ≤ 0 omitted from logarithmic plot



```
library(ggplot2)
ggplot(meancounts) + aes(control.mean, treated.mean) +
  geom_point(alpha=0.2) +
  scale_x_log10() +
  scale_y_log10()
```

Warning in scale_x_log10(): log-10 transformation introduced infinite values.

Warning in scale_y_log10(): log-10 transformation introduced infinite values.



We use log2 “fold-change” as a way to compare Log2 is easier to interpret than log10 because the output is easier to interpret.

```
# treated/control  
log2(10/10)
```

```
[1] 0
```

```
log2(20/10)
```

```
[1] 1
```

```
log10(20/10)
```

```
[1] 0.30103
```

```
log2(10/20)
```

```
[1] -1
```

```
log2(40/10)
```

```
[1] 2
```

Now lets add a row with the log2 FC:

```
meancounts$log2fc <- log2(meancounts$treated.mean/meancounts$control.mean)
head(meancounts)
```

	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000005	0.00	0.00	NaN
ENSG000000000419	520.50	546.00	0.06900279
ENSG000000000457	339.75	316.50	-0.10226805
ENSG000000000460	97.25	78.75	-0.30441833
ENSG000000000938	0.75	0.00	-Inf

A common “rule-of-thumb” threshold to say if something is up-regulated is a log2FC of $> +2$ or for down-regulated is log2FC < -2 .

There are 2 ways to deal with datasets that have 0 in it. One is Pseudocounts = add a small number to all genes. The better way is to just remove rows that have 0 in it because why would you want to analyze genes that aren’t detected anyways. Including them in the analysis will reduce the power of the analysis.

Filter out zero count genes

```
x <- c(1,5,0,5)
which(x==0)
```

```
[1] 3
```

```
y <- data.frame(a=c(1,5,0,5), b=c(1,0,5,5))
y==0
```

	a	b
[1,]	FALSE	FALSE
[2,]	FALSE	TRUE
[3,]	TRUE	FALSE
[4,]	FALSE	FALSE

```
which(y==0)
```

```
[1] 3 6
```

```
which(y==0, arr.ind=T)[,1]
```

```
[1] 3 2
```

only care about which gene (the row)

```
zero.inds <- which(meancounts[,1:2] == 0, arr.ind = T)[,1]  
mygenes <- meancounts[-zero.inds,]
```

Q8 How many genes are “up” regulated at the +2 log2FC threshold?

```
sum(mygenes$log2fc > 2)
```

```
[1] 250
```

Q9 How many genes are “down” regulated at the -2 log2FC threshold?

```
sum(mygenes$log2fc < -2)
```

```
[1] 367
```

Don't really trust this, we need statistics...

DESeq Analysis

Let's do this with DESeq2 and put some stats behind these numbers.

```
library(DESeq2)
```

DESeq wants 3 things for analysis, countData, colData, and design

```
dds <- DESeqDataSetFromMatrix(countData=counts,  
                              colData = metadata,  
                              design= ~dex)
```

converting counts to integer mode

Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in design formula are characters, converting to factors

The main function in the DESeq package to run analysis is called DESeq()

```
dds <- DESeq(dds)
```

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

Get the results out of this DESeq object with the function results()

```
res <- results(dds)
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 6 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.3507030	0.168246	-2.084470	0.0371175
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG0000000000419	520.134160	0.2061078	0.101059	2.039475	0.0414026
ENSG0000000000457	322.664844	0.0245269	0.145145	0.168982	0.8658106
ENSG0000000000460	87.682625	-0.1471420	0.257007	-0.572521	0.5669691
ENSG0000000000938	0.319167	-1.7322890	3.493601	-0.495846	0.6200029
	padj				
	<numeric>				

ENSG000000000003	0.163035
ENSG000000000005	NA
ENSG0000000000419	0.176032
ENSG0000000000457	0.961694
ENSG0000000000460	0.815849
ENSG0000000000938	NA

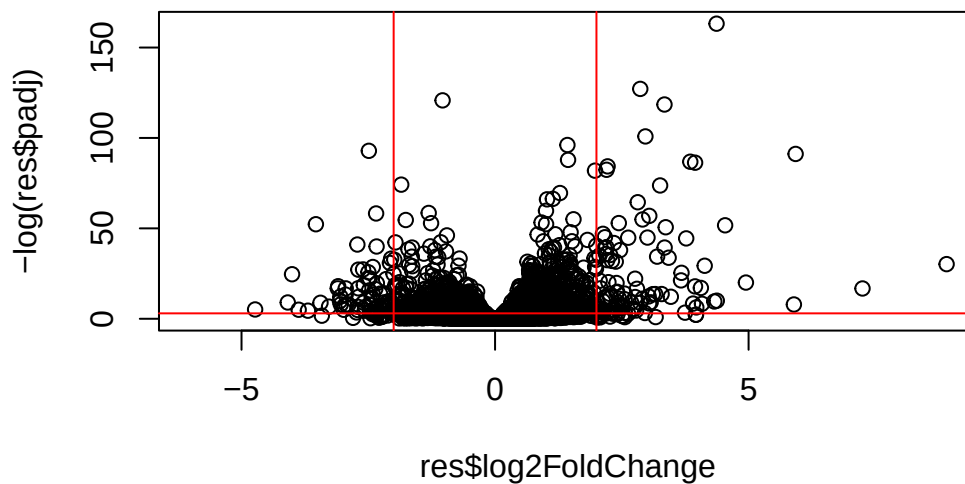
Save our results

```
write.csv(res, file="myresults.csv")
```

Volcano Plot

This is a plot of log2FC vs the adjusted p-value (x= log2FC, y=padj)

```
plot(res$log2FoldChange, -log(res$padj))
abline(v=c(-2,2), col="red")
abline(h=-log(0.05), col="red")
```

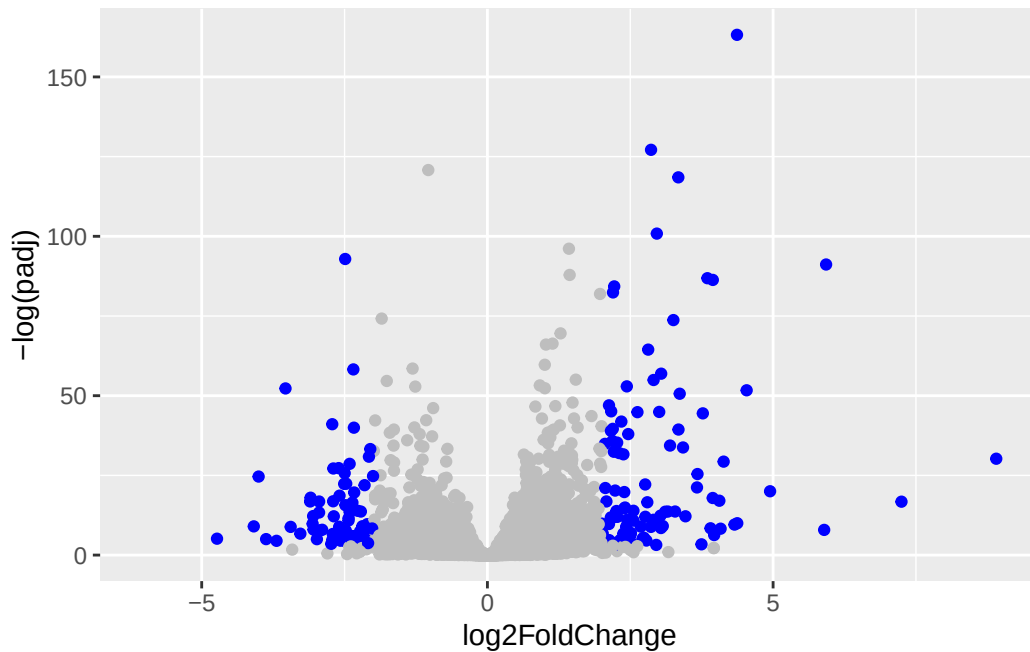


A nicer ggplot volcano plot

```
mycols <- rep("gray", nrow(res))
mycols[abs(res$log2FoldChange)>2] <- "blue"
mycols[res$padj >= 0.05] <- "grey"

ggplot(res) + aes(log2FoldChange, -log(padj)) +
  geom_point(col=mycols)
```

Warning: Removed 23549 rows containing missing values or values outside the scale range (`geom\_point()`).



Adding annotations

Our result table so far only contains the Ensembl gene IDs. However, alternative gene names and extra annotation are usually required for informative interpretation of our results. In this section we will add this necessary annotation data to our results.

We will use one of Bioconductor's main annotation packages to help with mapping between various ID schemes. Here we load the **AnnotationDbi** package and the annotation data package for humans **org.Hs.eg.db**.

```
library("AnnotationDbi")
library("org.Hs.eg.db")
```

columns are the kinds of things that can be retrieved from the database.

```
columns(org.Hs.eg.db)
```

```
[1] "ACCNUM"      "ALIAS"      "ENSEMBL"    "ENSEMBLPROT" "ENSEMBLTRANS"
[6] "ENTREZID"    "ENZYME"     "EVIDENCE"   "EVIDENCEALL"  "GENENAME"
[11] "GENETYPE"    "GO"         "GOALL"      "IPI"          "MAP"
[16] "OMIM"        "ONTOLOGY"   "ONTOLOGYALL" "PATH"         "PFAM"
[21] "PMID"        "PROSITE"    "REFSEQ"     "SYMBOL"       "UCSCCKG"
[26] "UNIPROT"
```

The main function we will use from the **AnnotationDbi** package is called `mapIds()`.

We can use the `mapIds()` function to add individual columns to our results table. We provide the row names of our results table as a key, and specify that `keytype=ENSEMBL`. The `column` argument tells the `mapIds()` function which information we want, and the `multiVals` argument tells the function what to do if there are multiple possible values for a single input value. Here we ask to just give us back the first one that occurs in the database.

```
res$symbol <- mapIds(org.Hs.eg.db,
                     keys=row.names(res), # Our genenames
                     keytype="ENSEMBL",   # The format of our genenames
                     column="SYMBOL",     # The new format we want to add
                     multiVals="first")
```

'select()' returned 1:many mapping between keys and columns

```
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 7 columns

```
      baseMean log2FoldChange      lfcSE      stat      pvalue
<numeric>      <numeric> <numeric> <numeric> <numeric>
```

ENSG000000000003	747.194195	-0.3507030	0.168246	-2.084470	0.0371175
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG0000000000419	520.134160	0.2061078	0.101059	2.039475	0.0414026
ENSG0000000000457	322.664844	0.0245269	0.145145	0.168982	0.8658106
ENSG0000000000460	87.682625	-0.1471420	0.257007	-0.572521	0.5669691
ENSG0000000000938	0.319167	-1.7322890	3.493601	-0.495846	0.6200029
	padj	symbol			
	<numeric>	<character>			
ENSG000000000003	0.163035	TSPAN6			
ENSG000000000005	NA	TNMD			
ENSG0000000000419	0.176032	DPM1			
ENSG0000000000457	0.961694	SCYL3			
ENSG0000000000460	0.815849	FIRRM			
ENSG0000000000938	NA	FGR			

Q11. Run the `mapIds()` function two more times to add the ENtrez ID and UniProt accession and GENENAME as new columns called `res$entrez`, `res$uniprot`, and `res$genename`.

```
res$entrez <- mapIds(org.Hs.eg.db,
                     keys=row.names(res),
                     column="ENTREZID",
                     keytype="ENSEMBL",
                     multiVals="first")
```

'select()' returned 1:many mapping between keys and columns

```
res$uniprot <- mapIds(org.Hs.eg.db,
                     keys=row.names(res),
                     column="UNIPROT",
                     keytype="ENSEMBL",
                     multiVals="first")
```

'select()' returned 1:many mapping between keys and columns

```
res$genename <- mapIds(org.Hs.eg.db,
                      keys=row.names(res),
                      column="GENENAME",
                      keytype="ENSEMBL",
                      multiVals="first")
```

'select()' returned 1:many mapping between keys and columns

```
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 10 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.3507030	0.168246	-2.084470	0.0371175
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.2061078	0.101059	2.039475	0.0414026
ENSG000000000457	322.664844	0.0245269	0.145145	0.168982	0.8658106
ENSG000000000460	87.682625	-0.1471420	0.257007	-0.572521	0.5669691
ENSG000000000938	0.319167	-1.7322890	3.493601	-0.495846	0.6200029
	padj	symbol	entrez	uniprot	
	<numeric>	<character>	<character>	<character>	
ENSG000000000003	0.163035	TSPAN6	7105	AOA087WYV6	
ENSG000000000005	NA	TNMD	64102	Q9H2S6	
ENSG000000000419	0.176032	DPM1	8813	H0Y368	
ENSG000000000457	0.961694	SCYL3	57147	X6RHX1	
ENSG000000000460	0.815849	FIRRM	55732	A6NFP1	
ENSG000000000938	NA	FGR	2268	B7Z6W7	
	genename				
	<character>				
ENSG000000000003	tetraspanin 6				
ENSG000000000005	tenomodulin				
ENSG000000000419	dolichyl-phosphate m..				
ENSG000000000457	SCY1 like pseudokina..				
ENSG000000000460	FIGNL1 interacting r..				
ENSG000000000938	FGR proto-oncogene, ..				

You can arrange and view the results by the adjusted p-value

```
ord <- order( res$padj )  
head(res[ord,])
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 10 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
--	----------	----------------	-------	------	--------

	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG00000152583	954.771	4.36836	0.2371268	18.4220	8.74490e-76
ENSG00000179094	743.253	2.86389	0.1755693	16.3120	8.10784e-60
ENSG00000116584	2277.913	-1.03470	0.0650984	-15.8944	6.92855e-57
ENSG00000189221	2383.754	3.34154	0.2124058	15.7319	9.14433e-56
ENSG00000120129	3440.704	2.96521	0.2036951	14.5571	5.26424e-48
ENSG00000148175	13493.920	1.42717	0.1003890	14.2164	7.25128e-46
	padj	symbol	entrez	uniprot	
	<numeric>	<character>	<character>	<character>	
ENSG00000152583	1.32441e-71	SPARCL1	8404	B4E2Z0	
ENSG00000179094	6.13966e-56	PER1	5187	A2I2P6	
ENSG00000116584	3.49776e-53	ARHGEF2	9181	A0A8Q3SIN5	
ENSG00000189221	3.46227e-52	MAOA	4128	B4DF46	
ENSG00000120129	1.59454e-44	DUSP1	1843	B4DRR4	
ENSG00000148175	1.83034e-42	STOM	2040	F8VSL7	
	genename				
	<character>				
ENSG00000152583	SPARC like 1				
ENSG00000179094	period circadian reg..				
ENSG00000116584	Rho/Rac guanine nucl..				
ENSG00000189221	monoamine oxidase A				
ENSG00000120129	dual specificity pho..				
ENSG00000148175	stomatin				

Finally, let's write out the ordered significant results with annotations. See the help for `?write.csv` if you are unsure here.

```
write.csv(res[ord,], "deseq_results.csv")
```

Pathway Analysis

Here we play with just one, the rather old (i.e. well established) GAGE package (which stands for Generally Applicable Gene set Enrichment), to do KEGG pathway enrichment analysis on our RNA-seq based differential expression results.

The KEGG pathway database, unlike GO for example, provides functional annotation as well as information about gene products that interact with each other in a given pathway, how they interact (e.g., activation, inhibition, etc.), and where they interact (e.g., cytoplasm, nucleus, etc.). Hence KEGG has the potential to provide extra insight beyond annotation lists of simple molecular function, process etc. from GO terms.

```
library(pathview)
```

```
#####  
Pathview is an open source software package distributed under GNU General  
Public License version 3 (GPLv3). Details of GPLv3 is available at  
http://www.gnu.org/licenses/gpl-3.0.html. Particullary, users are required to  
formally cite the original Pathview paper (not just mention it) in publications  
or products. For details, do citation("pathview") within R.
```

```
The pathview downloads and uses KEGG data. Non-academic uses may require a KEGG  
license agreement (details at http://www.kegg.jp/kegg/legal.html).  
#####
```

```
library(gage)
```

```
library(gageData)  
data(kegg.sets.hs)
```

```
#examine the first 2 pathways in this kegg set for humans  
head(kegg.sets.hs, 2)
```

```
$`hsa00232 Caffeine metabolism`
```

```
[1] "10" "1544" "1548" "1549" "1553" "7498" "9"
```

```
$`hsa00983 Drug metabolism - other enzymes`
```

```
[1] "10" "1066" "10720" "10941" "151531" "1548" "1549" "1551"  
[9] "1553" "1576" "1577" "1806" "1807" "1890" "221223" "2990"  
[17] "3251" "3614" "3615" "3704" "51733" "54490" "54575" "54576"  
[25] "54577" "54578" "54579" "54600" "54657" "54658" "54659" "54963"  
[33] "574537" "64816" "7083" "7084" "7172" "7363" "7364" "7365"  
[41] "7366" "7367" "7371" "7372" "7378" "7498" "79799" "83549"  
[49] "8824" "8833" "9" "978"
```

The main `gage()` function requires a named vector of fold changes, where the names of the values are the Entrez gene IDs.

Note that we used the `mapIDs()` function above to obtain Entrez gene IDs (stored in `res$entrez`) and we have the fold change results from DESeq2 analysis (stored in `res$log2FoldChange`).

```
foldchanges = res$log2FoldChange
names(foldchanges) = res$entrez
head(foldchanges)
```

```

      7105      64102      8813      57147      55732      2268
-0.35070302      NA  0.20610777  0.02452695 -0.14714205 -1.73228897
```

Now, let's run the gage pathway analysis.

```
# Get the results
keggres = gage(foldchanges, gsets=kegg.sets.hs)
```

Now lets look at the object returned from gage().

```
attributes(keggres)
```

```
$names
[1] "greater" "less"    "stats"
```

It is a list with three elements, “greater”, “less” and “stats”.

You can also see this in your Environmnet panel/tab window of RStudio or use the R command `str(keggres)`.

Like any list we can use the dollar syntax to access a named element, e.g. `head(keggres$greater)` and `head(keggres$less)`.

Lets look at the first few down (less) pathway results:

```
# Look at the first three down (less) pathways
head(keggres$less, 3)
```

		p.geomean	stat.mean	p.val
hsa05332	Graft-versus-host disease	0.0004250461	-3.473346	0.0004250461
hsa04940	Type I diabetes mellitus	0.0017820293	-3.002352	0.0017820293
hsa05310	Asthma	0.0020045888	-3.009050	0.0020045888

		q.val	set.size	exp1
hsa05332	Graft-versus-host disease	0.09053483	40	0.0004250461
hsa04940	Type I diabetes mellitus	0.14232581	42	0.0017820293
hsa05310	Asthma	0.14232581	29	0.0020045888

